Online Resource 1: Operational Characterisation of the Primary Subject

Shweta Mishra[1*]

[1*]Independent Researcher, Moradabad, India.

Corresponding author(s). E-mail(s): shweta.mishra.research@gmail.com;

Abstract

Supplementary material accompanying "Boundary-Mutation Testing for Pattern-Based Secret Detection: A Rule-Level Method and Cross-Scanner Evaluation." This resource reports the operational behaviour of the primary subject's ingress pipeline, event queue, worker pool and fault tolerance. None of it is evidence for or against any claim in the main paper.

This document reports the operational behaviour of the primary subject: its ingress pipeline, event queue, worker pool and fault tolerance. **None of it is evidence for or against any claim in the main paper.** It is included for two reasons. First, a reader assessing whether a defect in this scanner is worth attending to is entitled to know whether the surrounding system is a maintained service or a prototype. Second, the measurements exist, were produced by the same pinned harness, and withholding them from a paper that argues for reporting inconvenient evidence would be inconsistent.

The unit of analysis in the main paper is a detection rule; here it is a running service. That is a different artefact, which is why this material is reported as a separate online resource and not as a research question.

1 Concurrency and latency

Fig. 1 gives throughput and tail latency for the circuit breaker from 1 to 128 threads, and Fig. 2 the ingress pipeline latency against payload size. Each thread performs a fixed 1,500 operations, so offered load grows with thread count; threads are released by a `threading.Barrier` so that timing begins only once every thread is running. Each level is repeated 7 times and reported with the 95% confidence interval of the mean.

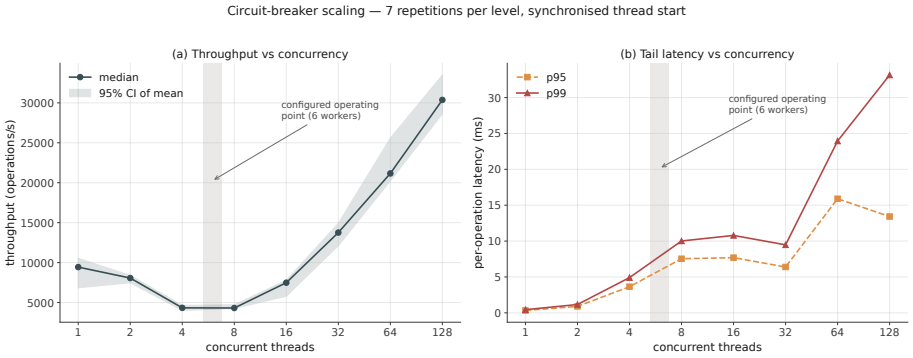

Fig. 1 Circuit-breaker throughput and tail latency, 1–128 threads, synchronised start, 7 repetitions per level.

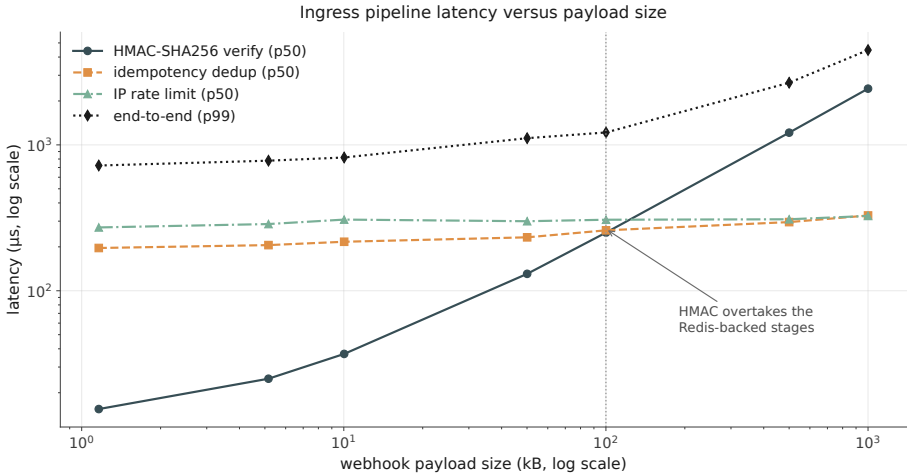

Fig. 2 Ingress pipeline latency against payload size, 400 trials per repetition, 5 repetitions.

The barrier matters, and its absence is the second harness defect we disclose. An earlier version started threads without synchronisation; at high thread counts the first threads finished before the last had started, and the resulting curve showed throughput *rising* with contention. That curve was an artefact of the harness, not a property of the system. We report it because it is the more instructive of our two mistakes: it produced a plausible, publishable-looking result that was entirely wrong.

Scanner throughput over 30 repetitions of 2,000 scans has a median of 11,114 scans/s (mean 11,104, sd 123), which is the basis for the claim in the main paper that the mutation battery is cheap to run. This figure, unlike every other number we report, is hardware-dependent: it is a wall-clock rate measured on the reference machine recorded in `environment/`, and a reproduction on other hardware should be expected to differ. It is quoted to establish an order of magnitude — the battery costs seconds, not hours — and no claim in this work depends on its exact value.

Table 1 Fault injection. All four oracles hold.

Fault	Oracle	Result
Redis killed mid-run	uncaught exceptions degraded gracefully	0 of 300 180 of 180
Redis unavailable	in-memory dedup preserved	holds
Duplicate delivery	exactly-once	1 of 10
Arrival storm	cap never exceeded	200 of 1,000

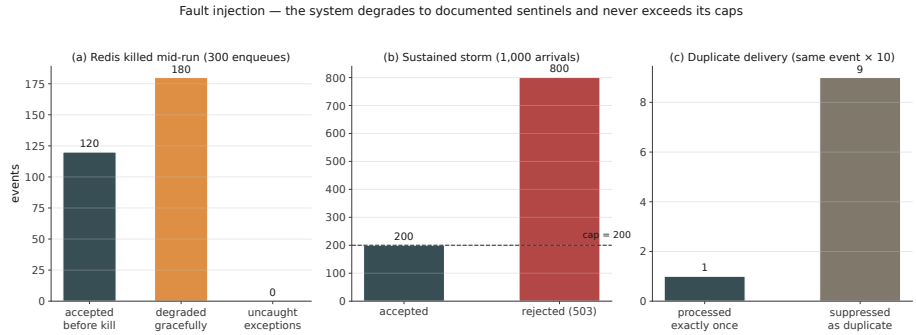

Fig. 3 Fault injection: Redis termination, dedup fallback, duplicate-delivery suppression, and arrival-storm capping.

2 Fault injection

Table 1 and Fig. 3 report four injected faults. All measurements use unmodified production modules against a real `redis-server` 7.0.15 on loopback with persistence disabled; the loopback configuration makes every latency figure a lower bound.

When Redis is terminated mid-run, `enqueue()` returns a sentinel rather than raising, so the caller falls back to direct dispatch: 120 events were accepted before the kill and the remaining 180 degraded without a single uncaught exception. Idempotency survives the loss of Redis through an in-memory fallback. Ten deliveries of one event yield exactly one processed and nine suppressed. Under a 1,000-event arrival storm against a queue capped at 200, exactly 200 are accepted and 800 rejected, with the cap never exceeded.

3 Test suite and complexity

Fig. 4 shows coverage by package and the cyclomatic-complexity distribution; the headline figures appear in the main paper's subject-characteristics table. The distribution is unremarkable — mean complexity 4.67 over 617 functions — with a single outlier at 34. We report it for completeness and draw no conclusion from it: nothing in the main paper depends on the subject's internal complexity, and the five defective rules are declarative table entries rather than control flow.

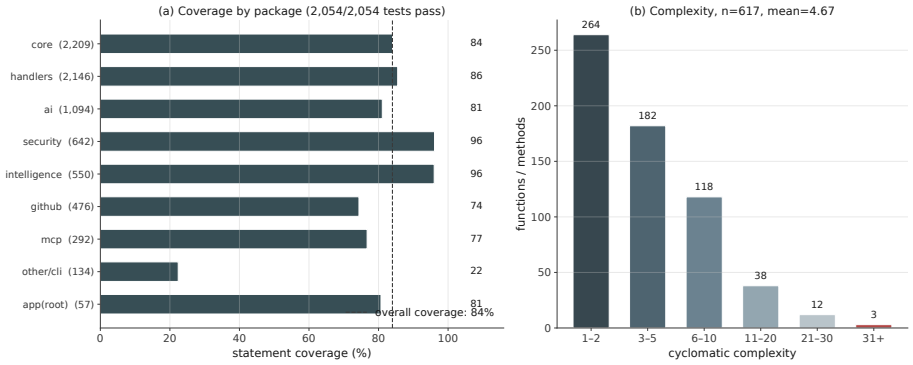

Fig. 4 Statement coverage by package and cyclomatic-complexity distribution for the primary subject.